Lab Ten – Point Estimation

Tips

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with "**Solution**:".
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use "enter" to make your code readable, so it doesn't run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you're submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Don't forget that you can copy-and-paste code when you're doing something similar as we did in class!
- Make sure to plot all computed probabilities. This is good `ggplot` practice AND will help make sure you don't make a mistake when computing the probability.

1 Lab 8 Notes

1.1 Altham's Multiplicative Binomial Distribution

The PMF for Altham's Multiplicative Binomial Distribution is

$$f_X(x) = \frac{1}{c}\binom{n}{x} p^x (1-p)^{n-x} \theta^{x(n-x)} I(x \in \{0, 1, ..., n\})$$

where

$$c = \sum_{i=0}^{n} \binom{n}{i} p^i (1-p)^{n-i} \theta^{i(n-i)}.$$

Just so we start on the same page, I have included my `R` function for computing probability using this distribution.

```
daltbinom <- function(x, size, prob, theta){
  normalizing.constant <- 0
  for(j in 0:size){
    normalizing.constant <- normalizing.constant +
      choose(size, j) * prob^j * (1-prob)^(size-j) * theta^(j*(size-j))
  }
  # Probability Mass at x=x
  (1/normalizing.constant) *
    choose(size, x) * prob^x * (1-prob)^(size-x) * theta^(x*(size-x)) *
      (x>=0)*(x<=size)
}
```

2 Altham's Multiplicative Beta Binomial Distribution

The PMF for Altham's Multiplicative Beta Binomial Distribution is

$$f_X(x) = \left[\int_0^1 \left(\frac{1}{c}\binom{n}{x} p^x (1-p)^{n-x} \theta^{x(n-x)}\right) \left(\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1}(1-p)^{\beta-1}\right) dp\right] I(x \in \{0, 1, ..., n\})$$

Just so we start on the same page, I have included my `R` function for computing probability using this distribution.

```r
daltbbinom.single <- function(x, size, alpha, beta, theta){
    integrand <- function(p) {
    sapply(p, # integrate will pass in a vector, so we use sapply to apply the following
           function(p_val){
             # f(x|p) * f(p|alpha, beta)
             daltbinom(x, size, p_val, theta) * dbeta(p_val, alpha, beta)
           }
    )
  }
  # Probability Mass at x=x
  # integrate(integrand, lower = 0, upper = 1)$value * (x>=0)*(x<=size)
  # I shrink the bounds below to avoid log(0)=-infinty
  # I use try() to catch when there is an error and avoid crashing
  res <- try(integrate(integrand, lower = 1e-7, upper = 1 - 1e-7)$value *
                 (x>=0)*(x<=size),
             silent = TRUE)
  # If there is an error, return a near-zero probability
  if(inherits(res, "try-error")) return(0.1^6)

  res
}
# Write a function that works on a vector
daltbbinom <- function(x, size, alpha, beta, theta){
  sapply(X=x, FUN=daltbbinom.single, size=size, alpha=alpha, beta=beta, theta=theta)
}
```

3 Question 1

In Lab 8, we use Altham's Multiplicative Binomial Distribution. Your job was to interpret the impact of θ in that equation. In this lab, we will model the number of legs won in a best of eleven (first to six) match.

To help us think about the interpretation, something that was challenging before, let's try something a little more manageable.

3.1 Part a

Suppose a player has a 50% chance of winning a specific leg, that is let $p = 0.50$. Let's look at the probability they win ($x = 6$) legs in the match. Compute this probability over a sequence from $\theta = 0.5$ to $\theta = 1.5$ and plot it using a line where the x-axis is θ and the y-axis is $P(X = 6)$.

Solution

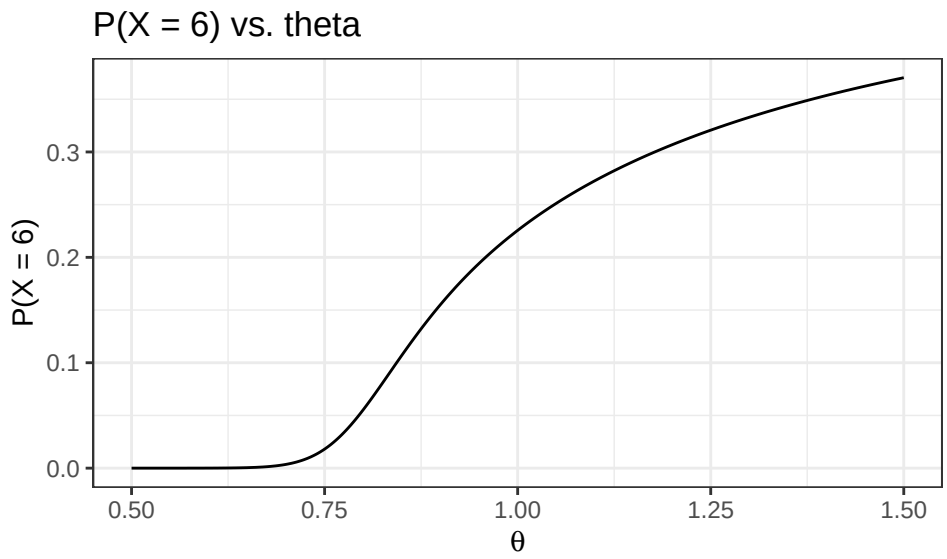
```r
#sequence of thetas
theta_seq <- seq(0.5, 1.5, length.out = 1000)

# probability of 6 wins
prob6 = daltbinom(6, 11, 0.5, theta_seq)

ggdat = tibble(theta = theta_seq, prob = prob6)
```

```r
ggplot(ggdat, aes(x = theta, y = prob)) +
  geom_line() +
  xlab(expression(theta)) +
  ylab("P(X = 6)") +
  ggtitle("P(X = 6) vs. theta") +
  theme_bw()
```

Figure 1: Altham's Binomial



3.2 Part b

Now, plot Altham's Multiplicative Binomial Distribution under the same parameters with $\theta = 0.50$, $\theta = 1$, and $\theta = 1.5$. Use `ncol=1` in `facet_wrap()` to plot the PMFs in a single column for comparing.

Solution

```
# prob for thetas
theta0.5 = daltbinom(6, 11, 0.5, 0.5)
theta1 = daltbinom(6, 11, 0.5, 1)
theta1.5 = daltbinom(6, 11, 0.5, 1.5)

ggdat = tibble(x = 0:11) |>
  mutate(theta0.5 = daltbinom(x, 11, 0.5, 0.50),
         theta1 = daltbinom(x, 11, 0.5, 1.00),
         theta1.5 = daltbinom(x, 11, 0.5, 1.50))
```

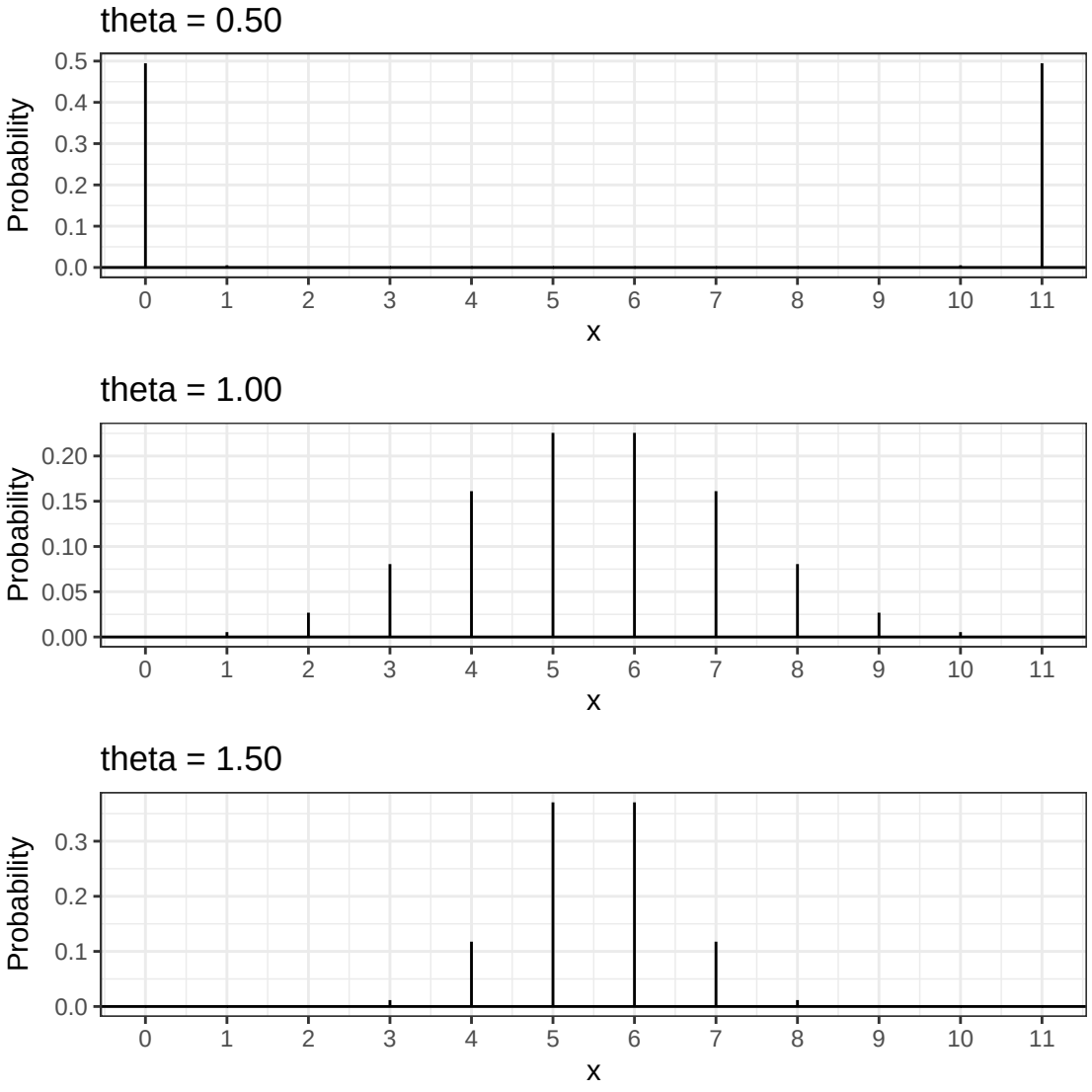
```
plot1 = ggplot() +
  geom_linerange(data = ggdat, aes(x = x, ymin = 0, ymax = theta0.5)) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:11) +
  ylab("Probability") +
  ggtitle("theta = 0.50") +
  theme_bw()

plot2 = ggplot() +
  geom_linerange(data = ggdat, aes(x = x, ymin = 0, ymax = theta1)) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:11) +
  ylab("Probability") +
  ggtitle("theta = 1.00") +
  theme_bw()

plot3 = ggplot() +
  geom_linerange(data = ggdat, aes(x = x, ymin = 0, ymax = theta1.5)) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks = 0:11) +
  ylab("Probability") +
  ggtitle("theta = 1.50") +
  theme_bw()

plot1 / plot2 / plot3
```

Figure 2: Altham's Binomial PMFs



3.3 Part c

What is your best assessment of what θ is doing in this setting. Consider when a 6-0 blowout is most likely, and when a 5-6 nail-biter is most likely. Note here we should treat the match as a fixed $n = 11$ as we did for $n = 3$ in Lab 8.
Note: In lab 8, we saw that small θ meant wins came in bunches (clumped together), so there were more 2-0 wins in a best of three series.

Solution A 6-0 blowout is most likely with a small theta, when wins come in clusters(hot hand). We can see on the theta = .5 graph how the extremeties are much more emphasized compared to a higher theta. A higher theta, like 1.5, is a lot more likely to end in a 6-5 score. We can see the 6 wins and 5 wins bars being the most probable on the theta = 1.5 graph because there is less hot hand effect.

4 Question 2

4.1 Part a

Altham's Multiplicative Binomial Distribution does not typically simplify to the clean $E(X) = np$ seen in the Binomial distribution, unless $\theta = 1$. Write out the expression that would need to be solved to find $E(X^k)$, and then write a function that computes it given the proposed parameter values p (`prob`) and θ (`theta`).

Solution

$E(X^k) = \sum_{i=-\infty}^{\infty} x^k * f_X(x)$

```
ealtbinom = function(k, size, prob,theta){
  x = 0:size
  sum(x^k * daltbinom(x, size, prob, theta))
}
# check to see if it equals mean of theta .5
ealtbinom(1,11,.5,.5)
```

```
[1] 5.5
```

4.2 Part b

Describe how you can use your answer to part a to find Method of Moments estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution Because we have two unknown parameters, we would need two equations in order to use a system of equations to find their values. We can use the expected value function from part a using k=1 and k=2 to get two equations using two estimators and solve for the values of p and theta that satisfy it.

4.3 Part c

Write a function that computes the log-likelihood for Altham's Multiplicative Binomial Distribution, given data (`x`) and proposed parameter values p (`prob`) and θ (`theta`).

Solution

```
llaltbinom = function(data = x, par, size, mle = F){
  prob = ifelse(mle, 1 / (1 + exp(-par[1])), par[1])
  theta = ifelse(mle, exp(par[2]), par[2])

  ll = sum(log(daltbinom(data,size,prob,theta)))

  ifelse(mle, -ll, ll)
}
```

4.4 Part d

Describe how you can use your answer to part c to find Maximum Likelihood estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution To find the mle I would use the optim function setting mle to true. This would contrain the search to 0-1 because of my if else statement and returns the negative log likelihood because optim minimizing by default but the min of the negative function would be the max.

5 Lab 10

Peter "Snakebite" Wright is a two-time Professional Darts Corporation World Champion (2020, 2022) and a former World No. 1. He is one of the sport's most decorated players, winning nearly 50 PDC titles including the World Matchplay, UK Open, and multiple European Championships.

In 2024, he made The Premier League – a league involving the eight best players that runs every Thursday night from February to May. In this season, he won two of eighteen matches, securing only four points. The spread in legs won was -43, meaning he lost 43 more legs than he won. This performance was rather quite bad. He was soundly trounced in almost all his match ups.

The 2024 standings are below:

Rank	Player
1	Luke Littler
2	Luke Humphries
3	Michael van Gerwen
4	Michael Smith
5	Nathan Aspinall
6	Rob Cross
7	Gerwyn Price
8	Peter Wright

5.1 Summarize the 2024 Data

In `pdc2024.csv`, I have provided the data for the 2024 Premier League season. There are a few data cleaning steps to consider.

- Remove any matches with no `Score` (e.g., a bye or walkover)
- Nights 1 through 16 are best of 11 (first to 6), but the playoffs are extended. For consistency, keep nights 1 through 16 only.
- Make two columns `WinnerLegs` and `LoserLegs` that keep track of the number of legs each player has won

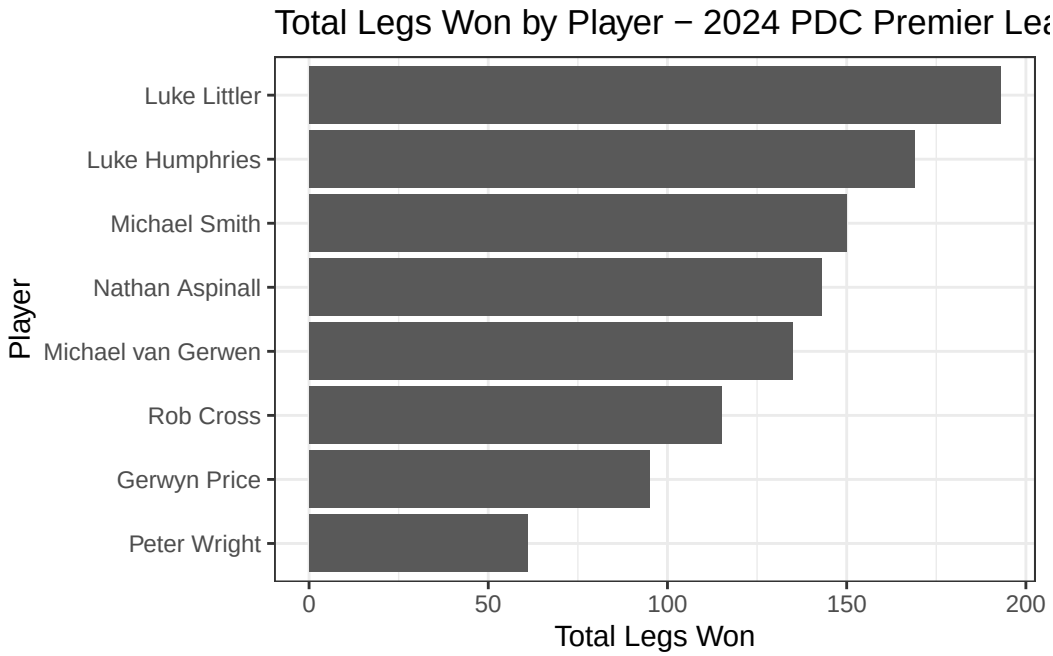
Solution

```
  pdcdata = read.csv("pdc2024.csv")
  dat.pdc.cleaned = pdcdata |>
     filter(Night != "Finals" & Score != "w/o") |>
     separate(col = "Score", into = c("WinnerLegs", "LoserLegs"))
```

Summarize the data. What do the data tell us about the breakdown of the results?

Solution

```
dat.pdc.cleaned |>
  mutate(
    WinnerLegs = as.numeric(WinnerLegs),
    LoserLegs  = as.numeric(LoserLegs)
  ) |>
  pivot_longer(
    cols = c(Winner, Loser),
    names_to = "Result",
    values_to = "Player"
  ) |>
  mutate(LegsWon = ifelse(Result == "Winner", WinnerLegs, LoserLegs)) |>
  group_by(Player) |>
  summarise(TotalLegs = sum(LegsWon)) |>
  ggplot(aes(x = reorder(Player, TotalLegs), y = TotalLegs)) +
  geom_col() +
  coord_flip() +
  xlab("Player") +
  ylab("Total Legs Won") +
  ggtitle("Total Legs Won by Player - 2024 PDC Premier League") +
  theme_bw()
```




```
dat.pdc.cleaned |>
  mutate(
    WinnerLegs = as.numeric(WinnerLegs),
    LoserLegs  = as.numeric(LoserLegs)
  ) |>
  pivot_longer(
    cols = c(Winner, Loser),
    names_to = "Result",
    values_to = "Player"
  ) |>
  mutate(LegsWon = ifelse(Result == "Winner", WinnerLegs, LoserLegs)) |>
  group_by(Player) |>
  summarise(
    mean   = mean(LegsWon),
    sd     = sd(LegsWon),
    median = median(LegsWon),
    IQR    = IQR(LegsWon),
    skew   = e1071::skewness(LegsWon),
    ekurt  = e1071::kurtosis(LegsWon)
  ) |>
  knitr::kable(digits = 3)
```

Player	mean	sd	median	IQR	skew	ekurt
Gerwyn Price	4.318	1.585	5	3	-0.435	-1.192
Luke Humphries	5.281	1.198	6	1	-1.623	1.466
Luke Littler	5.361	1.073	6	1	-1.402	0.413
Michael Smith	4.839	1.508	6	2	-0.921	-0.408
Michael van Gerwen	4.821	1.588	6	3	-0.788	-1.154
Nathan Aspinall	4.931	1.223	6	2	-0.440	-1.544
Peter Wright	3.389	1.614	4	2	-0.049	-1.230
Rob Cross	4.423	1.604	5	3	-0.509	-1.145

Summary On the graph and the mean of legs won by each player, we can see that the player with the most wins per match is not necessarily the winning player with Michael Smith and Nathan Aspinall winning more games than

Michael van Gerwen, and both having a higher mean games won, yet Michael van Gerwen had a higher rank, i.e. he won more matches.

5.2 Altham's Multiplicative Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maximium likelihood estimates for p and θ.
Note: The "for each" here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLE for the player using this data. Ensure to report p and θ for each player in a nicely formatted table.
  Note: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space. For example, here, you might consider `p <- 1/(1+exp(-par[1]))` and `theta <- exp(par[2])`.
- Make comments about the MLEs in the context of the standings.

Solution

```
dat.legs = dat.pdc.cleaned |>
  mutate(
    WinnerLegs = as.numeric(WinnerLegs),
    LoserLegs  = as.numeric(LoserLegs)
  ) |>
  pivot_longer(
    cols = c(Winner, Loser),
    names_to = "Result",
    values_to = "Player"
  ) |>
  mutate(LegsWon = ifelse(Result == "Winner", WinnerLegs, LoserLegs))

get.mle = function(player.legs) {
  fit = optim(
    par = c(0, 0),
    fn = function(par) llaltbinom(data = player.legs, par = par, size = 11, mle = TRUE)
  )
  list(
    p     = 1 / (1 + exp(-fit$par[1])),
    theta = exp(fit$par[2])
  )
}

players = unique(dat.legs$Player)

mle.results = tibble(
  Player = players,
  p      = NA_real_,
  theta  = NA_real_
)

for(i in 1:length(players)){
  player.legs = dat.legs |>
    filter(Player == players[i]) |>
    pull(LegsWon)

  fit = get.mle(player.legs)
  mle.results$p[i]     = fit$p
  mle.results$theta[i] = fit$theta
```

```r
}

mle.results |>
  knitr::kable(digits = 3)
```

Player	p	theta
Rob Cross	0.395	1.014
Peter Wright	0.317	0.989
Gerwyn Price	0.383	1.020
Nathan Aspinall	0.403	1.186
Michael Smith	0.426	1.047
Michael van Gerwen	0.431	1.023
Luke Littler	0.469	1.316
Luke Humphries	0.461	1.205

Comments The MLE estimates for p align closely with the final standings, higher ranked players tend to have higher estimated leg-winning probabilities, with Littler and Humphries at the top and Wright clearly last. The theta estimates suggest most players exhibit slight clustering (> 1), meaning matches tend toward closer scorelines, with Littler and Humphries showing the strongest tendency for closer matches and Peter Wright having the lowest.

5.3 Altham's Multiplicative Beta Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maxmium likelihood estimates for p and θ.
Note: The "for each" here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLEs for the player using this data. Ensure to report α, β, and θ for each player in a nicely formatted table.
  Note 1: Due to the computational complexity of `integrate()`, you should compute the logged probability for `0:6` once and add them according to the data in each player's vector. **Note 2**: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space.
- Plot the underlying Beta(α, β) distribution for each player.
  Note: Add the mean and variance of the Beta to the table of MLEs. Recall

$$E(X) = \frac{\alpha}{\alpha + \beta}$$
$$sd(X) = \sqrt{\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}}$$

- Make comments about the MLEs in the context of the standings.
  Note: The players alternate who throws first in a leg has the mathematical "head start" to reach a finish before their opponent can take another turn. Players alternate who throws first with each leg.

Solution

```r
llaltbbinom = function(data = x, par, size, mle = F){
  alpha = ifelse(mle, exp(par[1]), par[1])
  beta  = ifelse(mle, exp(par[2]), par[2])
  theta = ifelse(mle, exp(par[3]), par[3])

  log.probs = log(daltbbinom(0:size, size, alpha, beta, theta))

  ll = sum(log.probs[data + 1])

  ifelse(mle, -ll, ll)
```

```r
}

get.mle.bb = function(player.legs) {
  fit = optim(
    par = c(0, 0, 0),
    fn = function(par) llaltbbinom(data = player.legs, par = par, size = 11, mle = TRUE)
  )
  list(
    alpha = exp(fit$par[1]),
    beta  = exp(fit$par[2]),
    theta = exp(fit$par[3])
  )
}

mle.results.bb = tibble(
  Player = players,
  alpha  = NA_real_,
  beta   = NA_real_,
  theta  = NA_real_
)

for(i in 1:length(players)){
  player.legs = dat.legs |>
    filter(Player == players[i]) |>
    pull(LegsWon)

  fit = get.mle.bb(player.legs)
  mle.results.bb$alpha[i] = fit$alpha
  mle.results.bb$beta[i]  = fit$beta
  mle.results.bb$theta[i] = fit$theta
}

mle.results.bb |>
  mutate(
    mean = alpha / (alpha + beta),
    sd   = sqrt((alpha * beta) / ((alpha + beta)^2 * (alpha + beta + 1)))
  ) |>
  knitr::kable(digits = 3)
```

Player	alpha	beta	theta	mean	sd
Rob Cross	0.268	0.813	3.202	0.248	0.299
Peter Wright	1.786	7.289	1.214	0.197	0.125
Gerwyn Price	0.264	0.917	3.175	0.224	0.282
Nathan Aspinall	2.468	4.338	1.492	0.363	0.172
Michael Smith	0.534	1.068	2.245	0.334	0.292
Michael van Gerwen	0.219	0.454	5.001	0.325	0.362
Luke Littler	0.283	0.384	10.797	0.425	0.383
Luke Humphries	0.331	0.490	6.391	0.403	0.363

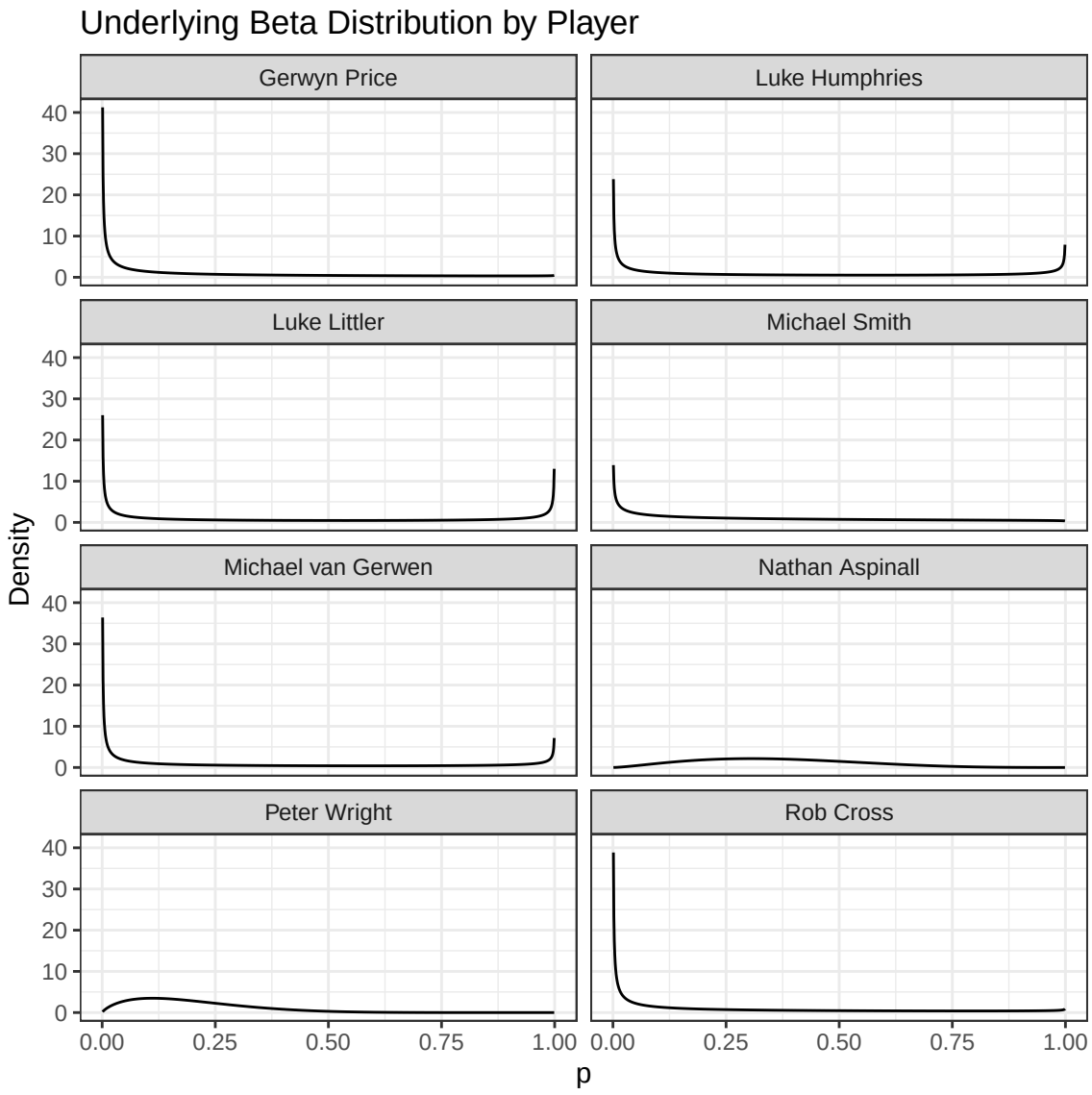
```r
p.seq = seq(0.001, 0.999, length.out = 1000)

ggdat.beta = mle.results.bb |>
  rowwise() |>
  mutate(density = list(tibble(p = p.seq,
                               d = dbeta(p.seq, alpha, beta)))) |>
  unnest(density)

ggplot(ggdat.beta, aes(x = p, y = d)) +
```

```
  geom_line() +
  facet_wrap(~ Player, ncol = 2) +
  xlab("p") +
  ylab("Density") +
  ggtitle("Underlying Beta Distribution by Player") +
  theme_bw()
```

Figure 3: Underlying Beta(alpha, Beta)



Comments The mean of the underlying Beta distribution follows the standings closely. Littler with a mean of 0.425 and Humphries with a mean of 0.403 rank the highest, while Wright, whose mean is 0.197, ranks last by a wide margin. The large theta estimates for top players like Littler (= 10.797) and Humphries (= 6.391) again suggest their matches strongly tend toward close alternating leg patterns. Wright's low Beta distribution (sd = 0.125) suggests his low leg-winning probability was consistent across matches.

5.4 Executive Summary

Summarize the work you've done here to provide a brief (200-300 word) data-driven summary of the 2024 Premier League season using your work in Lab 10. Your summary should be professional, clear, and all claims should be corroborated with statistical support.

Summary The 2024 PDC Premier League season featured eight of the world's best darts players competing across 16 nights of best-of-eleven (first to six legs) matches. Using Altham's Multiplicative Binomial and Altham's Multiplicative Beta Binomial distributions, we estimated each player's leg-winning probability p and clustering parameter theta through the MLE to better understand performance across the season. Looking at the raw data, total legs won by player shows a clear separation between the top and bottom of the standings. Littler and Humphries led in total legs and mean legs won per match(5.36 and 5.28 respectively), while Wright trailed behind by a wide margin. This is consistent with his 2 win finish. Notably, Michael Smith and Nathan Aspinall had higher mean legs won per match(4.84 and 4.931 respectively) than Michael van Gerwen(4.821), despite van Gerwen ranking above both in the end. This suggets he was more efficient at converting close matches into wins, or Nathan Aspinall and Michael Smith had close losses. We can confirm this theory by seeing the theta value for each player being >1 meaning closer matches, except for Peter Wright(.989) who lost many matches by a high differential. The Multiplicative Beta Binomial model adds further nuance by allowing each player's leg-winning probability to vary match-to-match via an underlying Beta distribution. Littler and Humphries had the highest estimated means (0.425 and 0.403 respectively), while Wright's underlying Beta mean near 0.197 with a standard deviation of 0.125, meaning his poor performance was not only low on average but consistently low across all matches. Littler and Humphries also had the largest theta hat estimates (10.797 and 6.391), suggesting their matches were especially prone to tight alternating-leg patterns.